

PROYECTO No. 2

Motor de Conversión de Autómatas Finitos No Deterministas (AFND) a Autómatas Finitos Deterministas (AFD) y Simulación

OBJETIVO GENERAL

Ampliar el sistema desarrollado en la Fase 1 para cargar y validar Autómatas Finitos No Deterministas (AFND), aplicar el algoritmo de construcción de subconjuntos y obtener un AFD equivalente que pueda visualizarse, analizarse y simularse mediante las funciones previamente implementadas.

OBJETIVOS DE APRENDIZAJE

- Representar formalmente AFD y AFND mediante la quintupla $M = (Q, \Sigma, \delta, q_0, F)$ y estructuras de datos apropiadas.
- Distinguir correctamente determinismo, no determinismo e incompletitud de la función de transición.
- Implementar el algoritmo de construcción de subconjuntos sin utilizar bibliotecas que realicen directamente la conversión.
- Documentar la relación entre los macroestados del AFD y los subconjuntos de estados del AFND original.
- Comprobar que el AFD generado reconoce el mismo lenguaje que el AFND de entrada.
- Reutilizar las funciones de carga, validación, análisis, simulación, trazas e historial desarrolladas en la Fase 1.

DESCRIPCIÓN DEL PROBLEMA

Este proyecto corresponde a la segunda fase del motor de autómatas. La solución no deberá reemplazar ni eliminar la funcionalidad anterior; deberá evolucionar el mismo programa para permitir la carga de AFND, su transformación en un AFD equivalente y la ejecución de las herramientas de la Fase 1 sobre el resultado.

Principio de compatibilidad: toda funcionalidad válida de la Fase 1 deberá continuar disponible tanto para un AFD cargado directamente como para el AFD generado desde un AFND.

Funcionalidad heredada de la Fase 1

La entrega deberá conservar operativas las siguientes capacidades:

- Crear o cargar un AFD manualmente y desde un archivo .txt.
- Mostrar la definición formal y la tabla de transición del AFD.
- Validar estados, alfabeto, estado inicial, estados finales y transiciones.

- Diferenciar un AFD válido y completo, un AFD incompleto, una definición no determinista y una definición inválida.
- Evaluar cadenas individuales y archivos de cadenas.
- Mostrar la traza paso a paso e indicar aceptación o rechazo.
- Conservar el historial de evaluaciones.
- Identificar estados alcanzables, inaccesibles y finales alcanzables, y determinar si el lenguaje podría ser vacío.
- Completar un AFD incompleto mediante un estado de trampa cuando el usuario lo solicite.

NUEVAS FUNCIONALIDADES DE LA FASE 2

Funcionalidad 1: Carga y validación de AFND

El menú principal deberá incorporar una opción para crear o cargar un AFND. El ingreso podrá realizarse de forma interactiva o mediante un archivo .txt. La representación formal conservará la quintupla $M = (Q, \Sigma, \delta, q_0, F)$, pero la función de transición podrá asociar un mismo par (estado, símbolo) con cero, uno o varios estados de llegada.

- Permitir múltiples estados destino para un mismo par (q, a) .
- Representar cada destino como un conjunto, aunque contenga un solo estado.
- Validar que todos los estados y símbolos utilizados hayan sido declarados.
- Distinguir una transición sin destinos ($\emptyset$) de una transición con uno o varios destinos.
- Mostrar la tabla de transición del AFND después de una carga válida.
- No se requieren transiciones ϵ en esta fase. Si aparecen, el sistema deberá informar que no forman parte del alcance solicitado.

Funcionalidad 2: Transformación de AFND a AFD

Después de cargar y validar el AFND, el sistema deberá aplicar automáticamente el algoritmo de construcción de subconjuntos para generar un AFD equivalente.

- El estado inicial del AFD será el macroestado $\{q_0\}$.
- Para cada macroestado S y cada símbolo $a \in \Sigma$, se calculará la unión de los destinos de todos los estados pertenecientes a S .
- Cada subconjunto nuevo deberá registrarse y procesarse hasta que no existan macroestados pendientes.
- El conjunto vacío $\emptyset$ se incorporará como estado de trampa cuando sea alcanzado durante la conversión.
- Un macroestado será final si contiene al menos un estado final del AFND original.
- El AFD resultante deberá tener exactamente un destino para cada par (macroestado, símbolo).

El sistema deberá mostrar una tabla de equivalencias que relacione cada identificador del AFD con el subconjunto del AFND que representa, por ejemplo:

Macroestado del AFD	Conjunto de estados del AFND
A	{q0}
B	{q0, q1}
C	$\emptyset$

Funcionalidad 3: Compatibilidad con las funciones de la Fase 1

Una vez generado el AFD equivalente, el sistema deberá almacenarlo en memoria utilizando la misma estructura de un AFD cargado directamente. Sobre el resultado deberán habilitarse:

- Definición formal y tabla de transición del AFD equivalente.
- Validación determinista y análisis estructural.
- Evaluación individual con traza paso a paso.
- Evaluación por lotes e historial de resultados.
- Detección de estados alcanzables e inaccesibles.
- Carga o creación de otro autómata sin reiniciar el programa.

FORMATO DEL ARCHIVO DE ENTRADA

El archivo deberá conservar una estructura analizable mediante expresiones regulares. Ejemplo de AFND:

```
NOMBRE=AFND_EJEMPLO
TIPO=AFND
ESTADOS=q0,q1,q2
ALFABETO=a,b
INICIAL=q0
FINALES=q2
TRANSICIONES:
q0,a,q0|q1
q0,b,q0
q1,a,∅
q1,b,q2
q2,a,q2
q2,b,q2
```

El separador “|” representa múltiples destinos y $\emptyset$ representa ausencia de destinos. El equipo podrá utilizar otra convención únicamente si la documenta, valida y aplica de forma consistente.

REQUISITOS FUNCIONALES

El programa deberá presentar un menú con, al menos, las siguientes opciones:

1. Crear un AFD manualmente.
2. Cargar un AFD desde un archivo .txt.
3. Crear un AFND manualmente.
4. Cargar un AFND desde un archivo .txt.
5. Mostrar la definición formal y la tabla del autómata cargado.
6. Validar la estructura del autómata.
7. Convertir el AFND cargado en un AFD equivalente.
8. Mostrar la tabla de equivalencias de macroestados.
9. Mostrar la tabla de transición del AFD generado.
10. Evaluar una cadena.
11. Evaluar un archivo de cadenas.
12. Consultar el historial de evaluaciones.
13. Realizar el análisis estructural.
14. Cargar o crear otro autómata.
15. Salir.

REGLAS DE OPERACIÓN

- No deberá permitirse una conversión si no existe un AFND válido cargado.
- No deberá permitirse evaluar cadenas sobre un AFND antes de completar la conversión, salvo que esa función se implemente como adicional.
- Después de la conversión, el usuario deberá poder consultar tanto el AFND original como el AFD equivalente.
- Al cargar un nuevo autómata, deberán limpiarse o separarse los datos, macroestados e historiales anteriores.
- Todos los errores deberán controlarse mediante mensajes comprensibles; el programa no deberá colapsar.

CASOS DE PRUEBA OBLIGATORIOS

- Un AFD válido y completo para comprobar la compatibilidad con la Fase 1.
- Un AFD incompleto para comprobar su clasificación y el uso del estado de trampa.
- Un AFND con dos destinos para un mismo par (estado, símbolo).
- Un AFND que genere al menos tres macroestados diferentes.
- Un AFND cuya conversión alcance el conjunto vacío $\emptyset$.
- Un AFND donde un macroestado final contenga estados finales y no finales.
- Una cadena aceptada, una rechazada, la cadena vacía y una cadena con símbolos fuera del alfabeto.
- Un archivo con errores sintácticos y otro con estados o símbolos inexistentes.
- Una comparación de resultados entre el AFND planteado y el AFD generado.

RESTRICCIONES

El proyecto deberá desarrollarse en Python mediante una aplicación de consola.

- No se permitirá utilizar bibliotecas que implementen directamente autómatas finitos o la construcción de subconjuntos.
- La representación deberá utilizar listas, conjuntos, diccionarios u otras estructuras justificadas.
- La función de transición no podrá implementarse mediante una cadena extensa de if/elif para un autómata específico.
- La solución deberá funcionar con cualquier AFD o AFND válido que respete el formato documentado.
- La lectura y validación sintáctica de archivos deberá utilizar expresiones regulares.
- El sistema deberá separar, al menos, carga, análisis sintáctico, validación, conversión y simulación.
- No se permitirá modificar el código fuente para cambiar el autómata evaluado.
- Las nuevas funciones deberán integrarse al programa de la Fase 1; no se aceptarán soluciones independientes sin continuidad funcional.

EVALUACIÓN

a. Documento PDF de análisis y diseño. Deberá incluir acciones, datos de entrada, variables, restricciones, expresiones regulares, estructuras de datos, algoritmo de construcción de subconjuntos, diagramas de flujo y casos de prueba con resultados esperados.

b. Programa funcional. Deberá integrar la Fase 1 y la Fase 2, controlar errores y estar documentado internamente con comentarios claros sobre la carga, validación, conversión y simulación.

Entregables y fecha:

- Documento de análisis y diseño en PDF.
- Manual de usuario.
- Manual técnico.
- Código fuente.
- Archivos de prueba.

La fecha y hora de presentación serán las indicadas en el Portal Académico.

CONSIDERACIONES IMPORTANTES

- El proyecto deberá desarrollarse en parejas conformadas por afinidad.
- La funcionalidad deberá ser implementada por los estudiantes; no se permite utilizar módulos realizados o compartidos por otros autores como sustitución del trabajo solicitado.

- Cada integrante deberá demostrar dominio completo de la solución para que se le asigne la nota obtenida.
- La entrega deberá realizarse únicamente en el espacio habilitado en el Portal Académico.

COPIA PARCIAL O TOTAL DEL PROYECTO TENDRÁ UNA NOTA DE 0 PUNTOS Y SE NOTIFICARÁ A LA COORDINACIÓN DEL ÁREA PARA LA APLICACIÓN DE LAS SANCIONES CORRESPONDIENTES.

POLÍTICA DE USO DE IA Y DOMINIO INDIVIDUAL DEL PROYECTO

El uso de herramientas de Inteligencia Artificial está permitido únicamente como guía de apoyo o consulta. No sustituye el análisis, diseño ni programación que debe realizar el estudiante.

Todo lo presentado deberá poder ser explicado, justificado y defendido individualmente. Cada integrante deberá explicar la arquitectura, las expresiones regulares, las estructuras de datos, el algoritmo de conversión y cualquier fragmento del código; además, deberá ejecutar el programa y realizar los cambios solicitados durante la defensa.

La falta de comprensión, la imposibilidad de justificar las decisiones implementadas o la dependencia total del compañero o de herramientas externas podrá calificarse como plagio o uso indebido de recursos, aplicándose una penalización individual de hasta el 100% de la nota según la gravedad del caso.

RÚBRICA DE EVALUACIÓN – CONVERSIÓN DE AFND A AFD Y SIMULACIÓN					
Criterio que se evalúa	Alcanzado	En Proceso alto	En Proceso bajo	Necesita mejorar	No se evidencia
Funcionalidad e integración de las fases (35 pts)	El programa conserva operativas las funciones de la Fase 1 e integra completamente la Fase 2: crea o carga AFD y AFND, valida y muestra ambos autómatas, convierte un AFND mediante construcción de subconjuntos, presenta la equivalencia de macroestados y permite analizar y simular el AFD generado. Todas las opciones funcionan sin fallos frecuentes. (35 pts)	Integra correctamente la Fase 1 y la mayoría de las funciones de la Fase 2. La carga, validación y conversión funcionan, pero existe una omisión menor en la visualización, consulta o reutilización del AFD generado. (28 pts)	Implementa el flujo principal, pero varias funciones están incompletas o presentan errores que afectan la carga de AFND, la conversión, la tabla de equivalencias o la compatibilidad con las funciones anteriores. (21 pts)	La solución funciona de forma limitada; conserva solo parte de la Fase 1 o realiza una conversión incompleta, específica para ciertos casos o con fallos frecuentes. (11 pts)	No presenta una implementación funcional, elimina la funcionalidad anterior o no permite cargar, validar y convertir un AFND en un AFD equivalente. (0 pts)
Manejo de errores y validación de entradas (15 pts)	Valida entradas manuales y archivos mediante expresiones regulares. Identifica secciones faltantes, formatos inválidos, estados o símbolos inexistentes, destinos múltiples, ausencia de destinos, transiciones fuera del alcance o operaciones ejecutadas en un orden no permitido. Informa errores comprensibles y, cuando corresponde, el número de líneas sin colarse. (15 pts)	Detecta y comunica correctamente la mayoría de los errores sintácticos, estructurales y de operación, aunque presenta pequeñas omisiones en la clasificación, el mensaje o la ubicación del error. (12 pts)	Realiza validaciones básicas, pero confunde o no controla varios casos de AFND, como destinos múltiples, conjunto vacío, símbolos no declarados, uso de ϵ o conversión sin un AFND válido. (8 pts)	Solo controla errores simples; entradas inválidas producen mensajes ambiguos, resultados incorrectos o interrupciones del programa. (5 pts)	No implementa las validaciones solicitadas o el programa colapsa ante archivos, transiciones u operaciones inválidas. (0 pts)
Diseño y codificación (10 pts)	El código es claro, modular y documentado. Separa carga, análisis sintáctico, validación, conversión y simulación; representa destinos y macroestados con estructuras apropiadas y reutiliza la misma estructura de AFD para autómatas cargados y generados. La solución es genérica y no utiliza bibliotecas que resuelvan directamente la conversión. (10 pts)	El diseño es modular y comprensible, utiliza estructuras de datos apropiadas y reutiliza la mayor parte de la lógica existente; presenta pequeñas inconsistencias, duplicación o acoplamiento menor. (8 pts)	El código es funcional pero parcialmente modular, mezcla responsabilidades, duplica lógica entre fases o utiliza de forma poco consistente conjuntos, listas y diccionarios. (6 pts)	El código es desordenado, difícil de mantener o depende de condiciones específicas para un autómata; existe poca continuidad técnica entre las dos fases. (3 pts)	No presenta código funcional o incumple los requisitos básicos de diseño, integración y codificación. (0 pts)
Documentación técnica y manuales (10 pts)	Entrega documentación completa y ordenada: quintupla de AFD y AFND, formato del archivo, expresiones regulares, estructuras de datos, validaciones, algoritmo de construcción de subconjuntos, tratamiento del conjunto vacío, diagramas, casos de prueba con resultados esperados, manual técnico y manual de usuario. (10 pts)	La documentación cubre casi todos los elementos requeridos y permite comprender, ejecutar y mantener el sistema, aunque contiene omisiones menores o explicaciones poco profundas. (8 pts)	La documentación es funcional pero incompleta; faltan varios elementos técnicos, diagramas, pruebas o instrucciones necesarias para comprender la conversión y el uso del sistema. (6 pts)	La documentación es deficiente, desordenada o poco clara y no respalda adecuadamente la implementación de las dos fases. (3 pts)	No presenta la documentación y los manuales solicitados, o el material es insuficiente para evaluar el proyecto. (0 pts)
CG2 – Liderazgo solidario y transformador (5 pts)	Se evidencia colaboración efectiva, coordinación, comunicación respetuosa, roles claros y distribución equitativa del trabajo entre la continuidad de la Fase 1 y las nuevas funciones. Todos los integrantes pueden explicar sus aportes y apoyar la solución integral. (5 pts)	Existe colaboración adecuada y participación de todos, con pequeños desequilibrios en responsabilidades o dominio de algunas partes. (4 pts)	La colaboración es irregular; algunos integrantes concentran tareas o muestran poco conocimiento de componentes desarrollados por el equipo. (3 pts)	La participación es mínima o muy desigual y se evidencia escasa coordinación entre los integrantes. (2 pts)	No se evidencia trabajo colaborativo o uno o más integrantes desconocen por completo el proyecto. (0 pts)
CG4 – Gestión del conocimiento técnico-científico (15 pts)	Explica y aplica correctamente los fundamentos de AFD y AFND: no determinismo, función de transición hacia conjuntos, construcción de subconjuntos, macroestados, estado inicial, estados finales, conjunto vacío como trampa y equivalencia de lenguajes. Justifica cada decisión de la conversión y su implementación. (15 pts)	Aplica correctamente la mayoría de los fundamentos y justifica las decisiones principales del algoritmo, con pequeñas imprecisiones que no afectan el AFD resultante. (12 pts)	La aplicación teórica es parcial; reconoce los conceptos principales, pero presenta dificultades para construir macroestados, determinar finales, tratar el conjunto vacío o justificar la equivalencia. (9 pts)	Menciona los conceptos de forma superficial y su relación con el código es débil o contiene errores importantes que afectan la conversión. (5 pts)	No se evidencia comprensión ni aplicación de los fundamentos teóricos requeridos para convertir un AFND en un AFD. (0 pts)
CE9 – Gestión estratégica de TI (10 pts)	Presenta un prototipo integrado, reutilizable y verificable: organiza el AFND y el AFD generado, muestra tablas de transición y equivalencias, conserva trazas e historial, ejecuta el análisis estructural y demuestra con los casos obligatorios que ambos autómatas reconocen el mismo lenguaje. (10 pts)	El prototipo integra y organiza adecuadamente la información; genera la mayoría de las salidas y pruebas requeridas, con fallos menores o una capacidad de análisis incompleta. (8 pts)	El prototipo cumple parcialmente su propósito; las tablas, trazas, análisis o comparación de resultados son limitados y dificultan verificar el AFD generado. (6 pts)	El prototipo es incompleto, presenta fallos frecuentes y aporta poca información útil para analizar la conversión o sus resultados. (3 pts)	No se evidencia un prototipo integrado ni una gestión adecuada de la información y las pruebas del proyecto. (0 pts)
Presentación y dominio individual (penalización)	Cada estudiante explica la arquitectura, las expresiones regulares, las estructuras de datos y el algoritmo de construcción de subconjuntos; ejecuta el programa, interpreta tablas y resultados y realiza cambios solicitados al código. Demuestra dominio total. Sin penalización. (0 pts)	Explica y ejecuta adecuadamente el proyecto, con leves dificultades para justificar una decisión o realizar un cambio menor. (-10 pts)	La explicación es incompleta; muestra dominio parcial, depende frecuentemente del equipo o tiene dificultades importantes para explicar o modificar la conversión. (-30 pts)	La explicación es deficiente y evidencia poco dominio de la arquitectura, la teoría, la lógica o el código implementado. (-60 pts)	No demuestra comprensión de lo implementado, no participa en la defensa o no puede ejecutar y explicar su aporte. (-90 pts)